

GLOBAL STORE

Retail Performance Analysis

An End-to-End SQL, Python & Power BI Analytics Project

Prepared by: Bilal Shafiq

Data Analyst | BI, SQL & Python

Dataset: Global Super Store (Kaggle) | Period: 2011 – 2014

Tech Stack: MySQL · Python (Pandas) · Power BI

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Business Context & Objectives	3
3. Methodology & Tools	4
4. Python — Data Acquisition & Cleaning	4
4.1 Notebook Workflow	4
4.2 Data Profiling	5
4.3 Cleaning & Feature Engineering	5
4.4 KPI & Business Analysis in Pandas	6
4.5 Profitability & Discount-Band Analysis	6
5. SQL — Database Design & Business Querying	7
5.1 Schema Design	7
5.2 Data Quality Checks	7
5.3 Executive KPI Query	9
5.4 Sales Analysis	10
5.5 Profit & Discount Analysis	13
5.6 Customer & Segment Analysis	15
5.7 Shipping & Operations Analysis	17
5.8 Seasonality Analysis	18
6. Power BI Dashboard	20
6.1 Executive Overview	20
6.2 Sales Performance	20
6.3 Profitability Analysis	21
6.4 Customer & Segment Analysis	21
6.5 Shipping & Operations	21
7. Key Findings	23
8. Business Recommendations	23
9. Conclusion	24

1. Executive Summary

Global Store operates across multiple markets, regions, product categories, and customer segments. This engagement analyzed four years of transaction-level data (2011–2014) to answer the questions that matter most to management: which products and regions drive revenue, where profit is being lost, how discounting affects margin, and what actions will improve profitable growth.

\$12.64M Total Sales	\$1.47M Total Profit	11.61% Profit Margin	25,035 Orders
1,590 Customers	10,292 Products	51,290 Transaction Rows	2011–2014 Date Range

Headline Finding

Revenue is not the problem — margin quality is. The business generates strong sales volume, but profit is uneven across products, discount levels, and regions. Three issues stand out:

- The Tables sub-category generates ~\$757K in sales but loses -\$64K — the clearest product-level problem in the dataset.
- Discounts above 30% turn a healthy 25.32% margin (no-discount orders) into a -51.27% margin — discounting is the single biggest controllable profit risk.
- Southeast Asia produces ~\$884K in sales but only ~\$17.9K in profit (≈2% margin) — a high-sales, low-margin region that needs review before further investment.

Recommended Actions

- Review Tables pricing, discount policy, supplier cost, and shipping cost.
- Cap or approve-only discounts above 30%, unless there is a clear inventory or strategic reason.
- Investigate Southeast Asia's product mix, discount behaviour, and shipping cost drivers.
- Grow Technology aggressively — it is the strongest category on both revenue and profit.
- Track revenue and profit margin together on the Power BI dashboard so growth decisions never reward unprofitable sales.

2. Business Context & Objectives

Revenue growth alone does not show whether a retail business is healthy. Discounts, shipping cost, product mix, and regional operating patterns can quietly erode profit even while top-line sales look strong. This project was scoped around seven commercial questions:

Area	Business Question
Executive KPIs	What are total sales, profit, orders, customers, quantity, and margin?
Sales	Which categories, products, regions, and markets generate the most revenue?
Profit	Which products and sub-categories create margin leakage?
Discounting	How does discount level affect profit margin?
Customers	Which segments and customers contribute most to revenue and profit?
Shipping	Which ship modes and priorities carry the highest cost?
Seasonality	Which months and quarters show stronger sales and profit patterns?

3. Methodology & Tools

The analysis follows a complete analytics pipeline: Python for data acquisition and cleaning, MySQL for relational modelling and business querying, and Power BI for executive-facing dashboards.

Tool	Role in this Project
Python (Pandas)	Data download, cleaning, transformation, feature engineering, CSV exports
MySQL	Relational schema design, data loading, quality checks, KPI and business queries
Power BI	Dashboard pages, KPI cards, trend visuals, product and executive reporting
Excel / CSV	Source data handling and processed table exports

Data Source

Dataset: Global Super Store Dataset (Kaggle) — <https://www.kaggle.com/datasets/apoorvaappz/global-super-store-dataset>

File	Description
data/raw/global_superstore_raw.csv	Raw Kaggle source file
data/processed/global_superstore_cleaned.csv	Cleaned transaction-level file
data/processed/customers.csv	Customer table for MySQL
data/processed/products.csv	Product table for MySQL
data/processed/orders.csv	Order table for MySQL
data/processed/sales.csv	Sales fact table for MySQL

4. Python — Data Acquisition & Cleaning

Python forms the data preparation layer of the project. Raw Kaggle data is profiled, cleaned, type-cast, enriched with derived fields, and exported into transaction-level and relational CSV tables ready for MySQL loading.

4.1 Notebook Workflow

Notebook	Purpose
01_data_understanding.ipynb	Dataset profiling — shape, dtypes, missing values
02_data_cleaning_and_preparation.ipynb	Type casting, deduplication, feature creation, CSV export
03_exploratory_data_analysis.ipynb	KPI calculation and category / product / trend checks
04_customer_and_segment_analysis.ipynb	Segment, region and top-customer performance
05_profitability_and_loss_analysis.ipynb	Margin leakage, loss-making products, discount bands
06_business_recommendations.ipynb	Insights converted into a structured action table

4.2 Data Profiling

The raw file is loaded and profiled for shape, data types, and missing values before any cleaning decisions are made:

```
from pathlib import Path
import pandas as pd

DATA_PATH = Path('../data/raw/global_superstore_raw.csv')
df = pd.read_csv(DATA_PATH, encoding='latin1')

pd.DataFrame({
    'column': df.columns,
    'dtype': df.dtypes.astype(str).values,
    'missing_values': df.isna().sum().values,
    'missing_pct': (df.isna().mean() * 100).round(2).values
})
```

4.3 Cleaning & Feature Engineering

Column names are standardized, dates and numeric fields are cast to the correct types, duplicate keys are checked, and derived business fields are created (profit margin, shipping days, and calendar attributes):

```
date_columns = ['Order_Date', 'Ship_Date']
for col in date_columns:
    df[col] = pd.to_datetime(df[col], format='%d-%m-%Y', errors='coerce')

numeric_columns = ['Sales', 'Profit', 'Discount', 'Shipping_Cost', 'Quantity']
for col in numeric_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')

df['Profit_Margin'] = df['Profit'] / df['Sales'].replace(0, pd.NA)
df['Shipping_Days'] = (df['Ship_Date'] - df['Order_Date']).dt.days
df['Year'] = df['Order_Date'].dt.year
df['Month'] = df['Order_Date'].dt.month
df['Quarter'] = df['Order_Date'].dt.quarter

CLEAN_PATH.parent.mkdir(parents=True, exist_ok=True)
df.to_csv(CLEAN_PATH, index=False)
```

Data quality checks confirm the file is fit for analysis before it is exported:

```
quality_checks = {
    'duplicate_row_id': df.duplicated(subset=['Row_ID']).sum(),
    'duplicate_order_id': df.duplicated(subset=['Order_ID']).sum(),
    'missing_values_total': int(df.isna().sum().sum())
}
```

4.4 KPI & Business Analysis in Pandas

With the cleaned file in hand, Python is used to compute the same executive KPIs, category performance, and trend views that are later validated in SQL and visualized in Power BI:

```
kpis = pd.Series({
    'Total Sales':      df['Sales'].sum(),
    'Total Profit':     df['Profit'].sum(),
    'Total Quantity':   df['Quantity'].sum(),
    'Profit Margin':    df['Profit'].sum() / df['Sales'].sum(),
    'Total Orders':     df['Order_ID'].nunique(),
    'Total Customers':  df['Customer_ID'].nunique()
})

category_sales = df.groupby('Category', as_index=False)['Sales'].sum()
                    .sort_values('Sales', ascending=False)
sns.barplot(data=category_sales, x='Sales', y='Category')
plt.title('Sales by Category')
```

4.5 Profitability & Discount-Band Analysis

The discount-band logic — the single strongest driver of margin risk identified in this project — is built directly in Pandas:

```
discount_profit = df.copy()
discount_profit['Discount_Band'] = pd.cut(
    discount_profit['Discount'],
    bins=[-0.01, 0, 0.1, 0.2, 0.3, 1],
    labels=['No discount', '0-10%', '11-20%', '21-30%', 'Above 30%']
)
discount_profit.groupby('Discount_Band', observed=True).agg(
    {'Sales': 'sum', 'Profit': 'sum'}
).assign(Profit_Margin=lambda x: x['Profit'] / x['Sales'])
```

This Python-side aggregation matches the SQL discount-band query in Section 5.5, cross-validating the finding that discounts above 30% push margin negative.

5. SQL — Database Design & Business Querying

A normalized MySQL database (global_store) was built from the Python-processed CSV exports. All business queries run against a single reporting view, vw_global_store_analysis, which joins customers, orders, products, and sales into one denormalized table carrying order dates, shipping days, customer segment, product hierarchy, region/market, and all financial measures.

5.1 Schema Design

```
CREATE DATABASE IF NOT EXISTS global_store;
USE global_store;

CREATE TABLE customers (
  customer_id  VARCHAR(50) PRIMARY KEY,
  customer_name VARCHAR(255),
  segment     VARCHAR(100)
);

CREATE TABLE products (
  product_id   VARCHAR(50) PRIMARY KEY,
  product_name VARCHAR(1000),
  category     VARCHAR(100),
  sub_category VARCHAR(100)
);

-- orders and sales tables follow the same pattern,
-- linked by customer_id / product_id / order_id
```

Table	Rows Loaded
customers	1,590
products	10,292
orders	25,035
sales	51,290

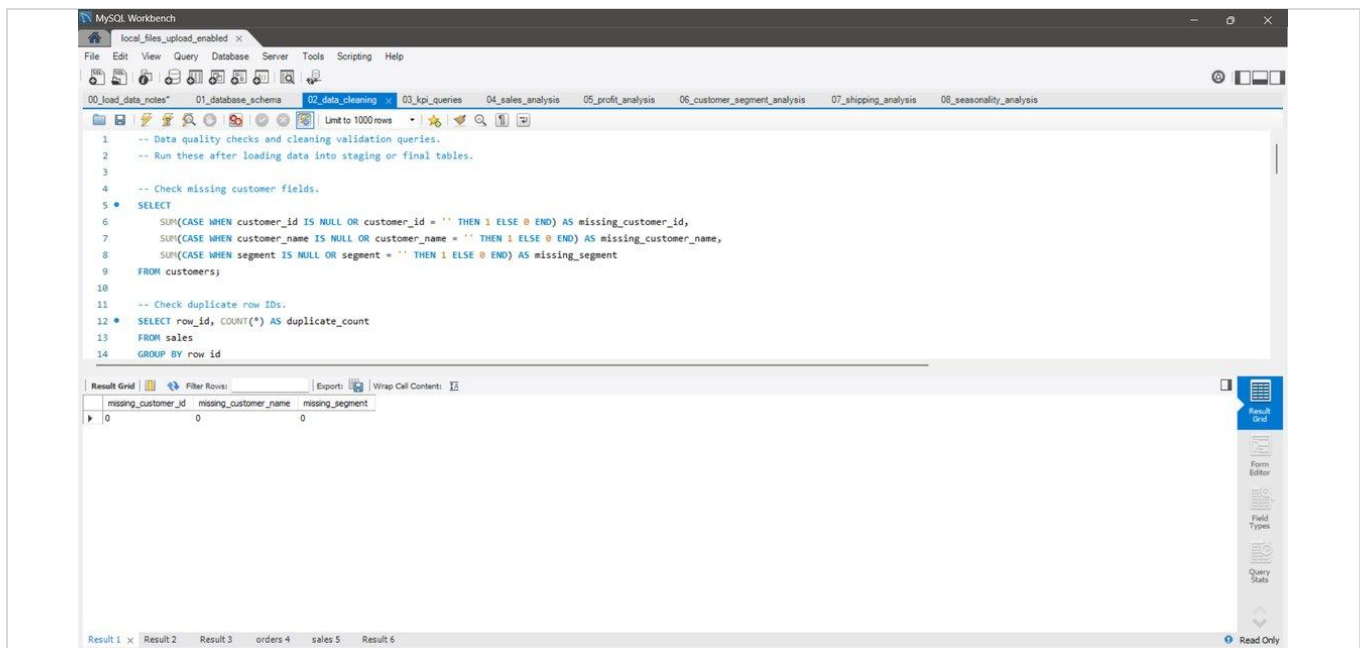
5.2 Data Quality Checks

Before any business analysis, the loaded tables are validated for missing keys, duplicate rows, invalid dates, and out-of-range numeric values:

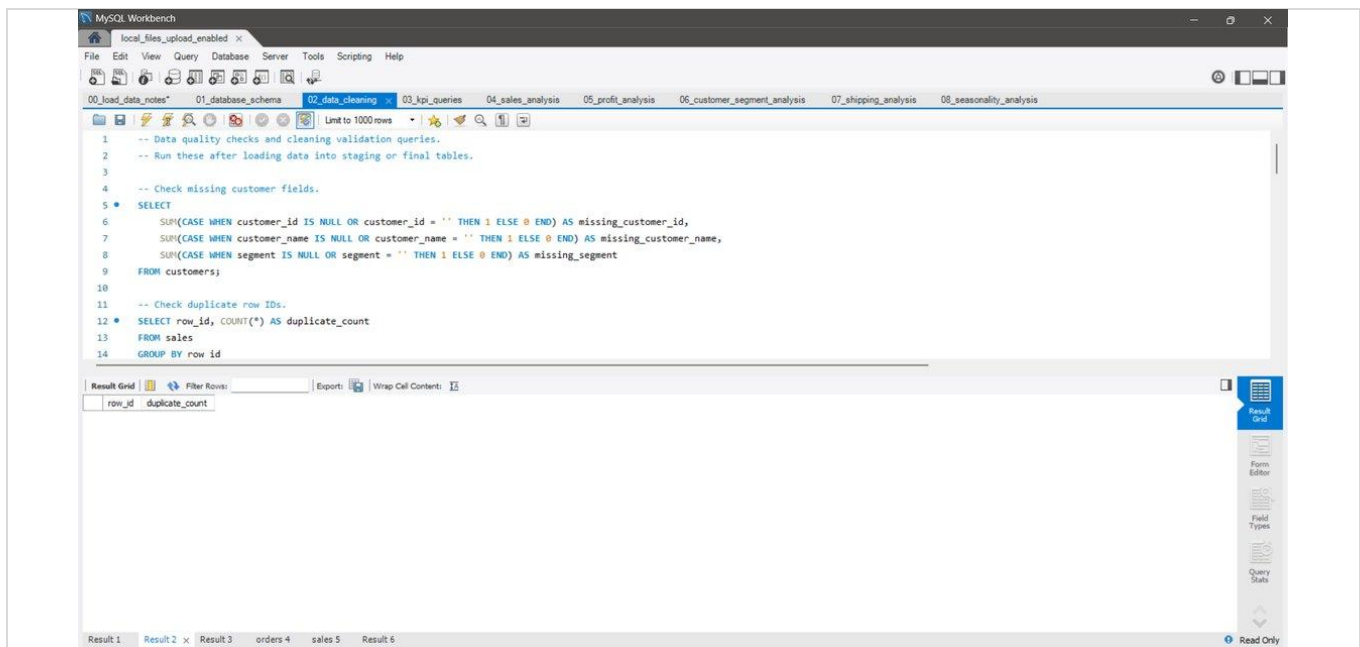
```
-- Check duplicate row IDs
SELECT row_id, COUNT(*) AS duplicate_count
FROM sales
GROUP BY row_id
HAVING COUNT(*) > 1;

-- Validate date sequence
SELECT order_id, order_date, ship_date
FROM orders
WHERE ship_date < order_date;

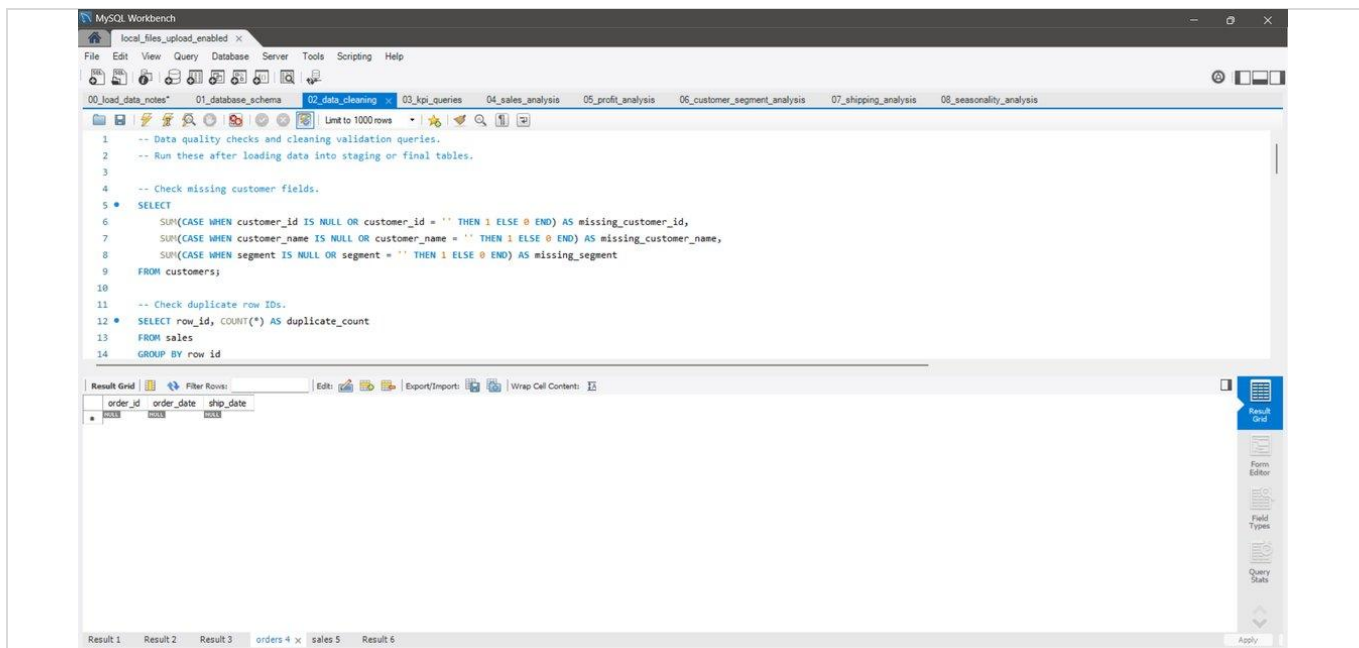
-- Check invalid numeric values
SELECT * FROM sales
WHERE sales < 0 OR quantity <= 0
      OR shipping_cost < 0 OR discount > 1;
```



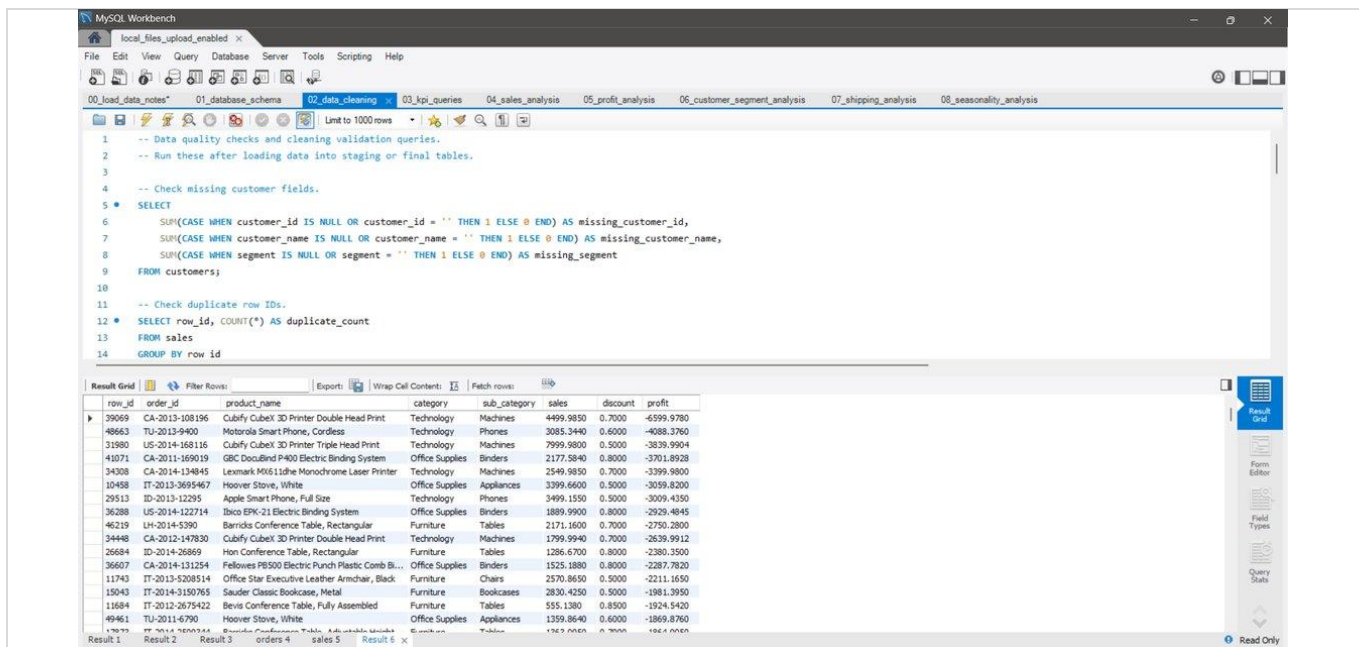
MySQL Workbench — missing customer field check



MySQL Workbench — duplicate row ID check (0 duplicates found)



MySQL Workbench — ship-date vs order-date sequence validation



MySQL Workbench — negative-profit record inspection

5.3 Executive KPI Query

A single query on the reporting view produces the executive KPI set used across the whole report and the Power BI overview page:

```

SELECT
  ROUND(SUM(sales), 2) AS total_sales,
  ROUND(SUM(profit), 2) AS total_profit,
  SUM(quantity) AS total_quantity,
  ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin,
  COUNT(DISTINCT order_id) AS total_orders,
  COUNT(DISTINCT customer_id) AS total_customers,

```

```

ROUND(SUM(sales) / NULLIF(COUNT(DISTINCT order_id), 0), 2) AS average_order_value,
ROUND(AVG(shipping_cost), 2) AS average_shipping_cost,
ROUND(AVG(discount), 4) AS average_discount_rate
FROM vw_global_store_analysis;

```

MySQL Workbench — executive KPI output

	total_sales	total_profit	total_quantity	profit_margin	total_orders	total_customers	average_order_value	average_shipping_cost	average_discount_rate
Result 1	12642501.91	1467457.29	178312	0.1161	25035	1589	504.99	26.38	0.1429

MySQL Workbench — executive KPI output

MySQL Workbench — count of loss-making products

	loss_making_products
Result 2	2889

MySQL Workbench — count of loss-making products

5.4 Sales Analysis

Sales are broken down by product, category, sub-category, region, market, and month to identify revenue drivers:

```

SELECT category, ROUND(SUM(sales), 2) AS total_sales
FROM vw_global_store_analysis

```

```
GROUP BY category
ORDER BY total_sales DESC;
```

The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query is as follows:

```

1  -- Sales performance analysis.
2
3  -- Top 10 products by sales.
4  * SELECT product_name, ROUND(SUM(sales), 2) AS total_sales
5  FROM vw_global_store_analysis
6  GROUP BY product_name
7  ORDER BY total_sales DESC
8  LIMIT 10;
9
10 -- Sales by category.
11 * SELECT category, ROUND(SUM(sales), 2) AS total_sales
12 FROM vw_global_store_analysis
13 GROUP BY category
14 ORDER BY total_sales DESC;
```

The result grid shows the following data:

category	total_sales
Technology	4744557.50
Furniture	4110874.19
Office Supplies	3787070.23

MySQL Workbench — sales by category

The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query is as follows:

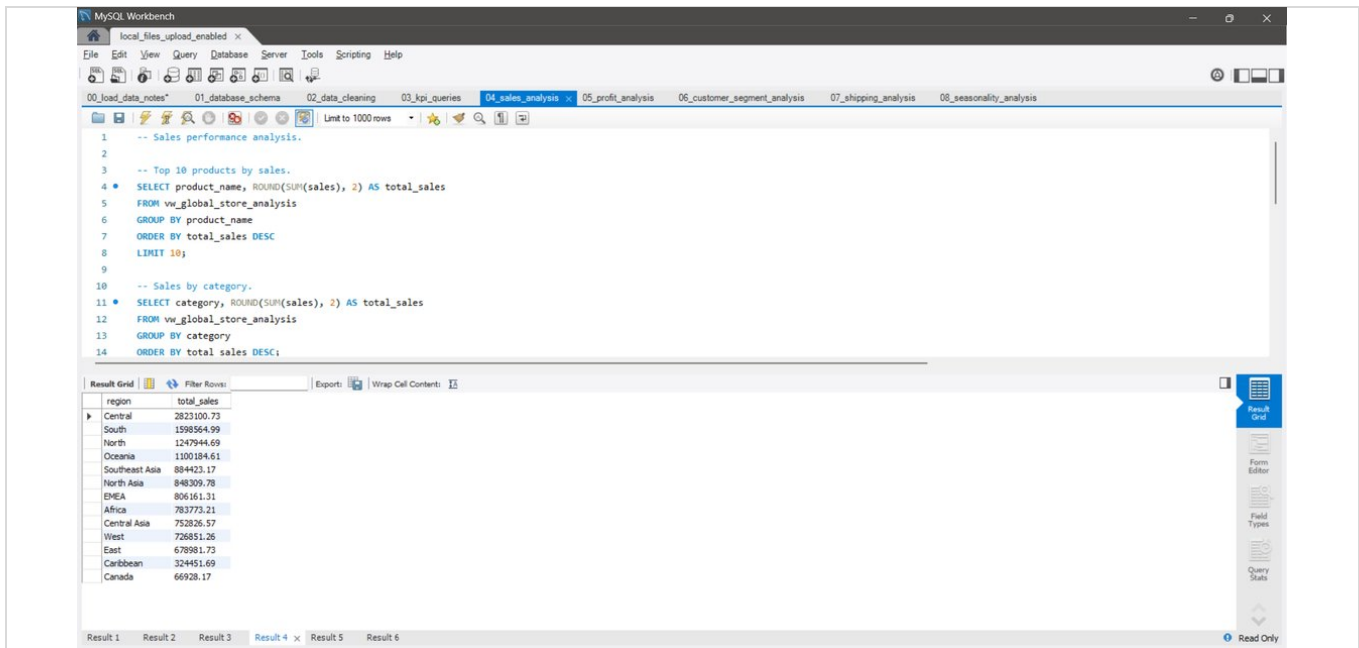
```

1  -- Sales performance analysis.
2
3  -- Top 10 products by sales.
4  * SELECT product_name, ROUND(SUM(sales), 2) AS total_sales
5  FROM vw_global_store_analysis
6  GROUP BY product_name
7  ORDER BY total_sales DESC
8  LIMIT 10;
9
10 -- Sales by category.
11 * SELECT category, ROUND(SUM(sales), 2) AS total_sales
12 FROM vw_global_store_analysis
13 GROUP BY category
14 ORDER BY total_sales DESC;
```

The result grid shows the following data:

product_name	total_sales
Apple Smart Phone, Full Size	86935.78
Cisco Smart Phone, Full Size	76441.53
Motorola Smart Phone, Full Size	73156.30
Nokia Smart Phone, Full Size	71904.56
Canon imageCLASS 2200 Advanced Copier	61599.82
Hon Executive Leather Armchair, Adjustable	58193.48
Office Star Executive Leather Armchair, Adjustable	51449.80
Harbour Crest Executive Leather Armchair, Adjustable	50121.52
Samsung Smart Phone, Cordless	48653.46
Nokia Smart Phone, with Caller ID	47877.79

MySQL Workbench — top 10 products by sales



MySQL Workbench

local_files_upload_enabled x

File Edit View Query Database Server Tools Scripting Help

00_load_data_notes 01_database_schema 02_data_cleaning 03_kpi_queries 04_sales_analysis 05_profit_analysis 06_customer_segment_analysis 07_shipping_analysis 08_seasonality_analysis

Limit to 1000 rows

```

1 -- Sales performance analysis.
2
3 -- Top 10 products by sales.
4 * SELECT product_name, ROUND(SUM(sales), 2) AS total_sales
5 FROM vw_global_store_analysis
6 GROUP BY product_name
7 ORDER BY total_sales DESC
8 LIMIT 10;
9
10 -- Sales by category.
11 * SELECT category, ROUND(SUM(sales), 2) AS total_sales
12 FROM vw_global_store_analysis
13 GROUP BY category
14 ORDER BY total_sales DESC;

```

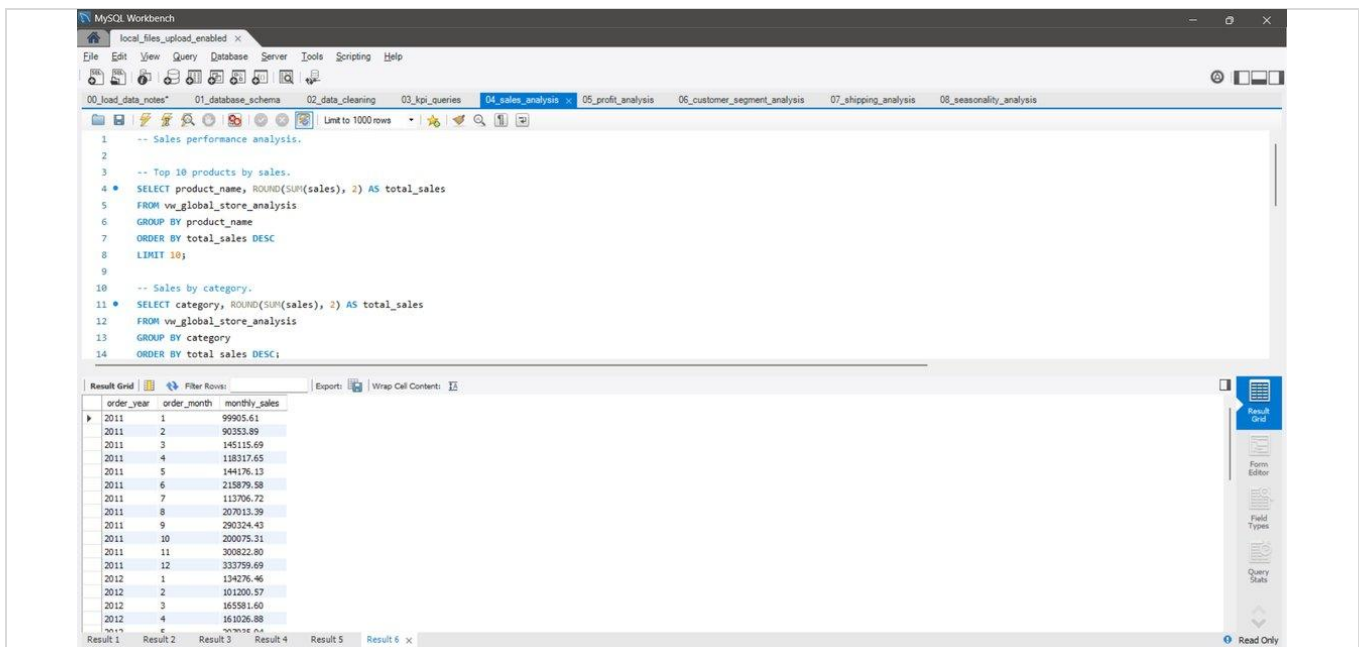
Result Grid

region	total_sales
Central	2823100.73
South	1598564.99
North	1247944.69
Oceania	1100184.61
Southeast Asia	884423.17
North Asia	848309.78
EMEA	806161.31
Africa	783773.21
Central Asia	752826.57
West	726851.26
East	678991.73
Caribbean	324451.69
Canada	66928.17

Result 1 Result 2 Result 3 Result 4 x Result 5 Result 6

Read Only

MySQL Workbench — sales by region



MySQL Workbench

local_files_upload_enabled x

File Edit View Query Database Server Tools Scripting Help

00_load_data_notes 01_database_schema 02_data_cleaning 03_kpi_queries 04_sales_analysis 05_profit_analysis 06_customer_segment_analysis 07_shipping_analysis 08_seasonality_analysis

Limit to 1000 rows

```

1 -- Sales performance analysis.
2
3 -- Top 10 products by sales.
4 * SELECT product_name, ROUND(SUM(sales), 2) AS total_sales
5 FROM vw_global_store_analysis
6 GROUP BY product_name
7 ORDER BY total_sales DESC
8 LIMIT 10;
9
10 -- Sales by category.
11 * SELECT category, ROUND(SUM(sales), 2) AS total_sales
12 FROM vw_global_store_analysis
13 GROUP BY category
14 ORDER BY total_sales DESC;

```

Result Grid

order_year	order_month	monthly_sales
2011	1	99905.61
2011	2	90353.89
2011	3	145115.69
2011	4	118317.65
2011	5	144176.13
2011	6	218879.58
2011	7	113706.72
2011	8	207013.39
2011	9	290324.43
2011	10	200075.31
2011	11	300822.80
2011	12	333759.69
2012	1	134276.46
2012	2	101200.57
2012	3	165581.60
2012	4	161026.88

Result 1 Result 2 Result 3 Result 4 Result 5 Result 6 x

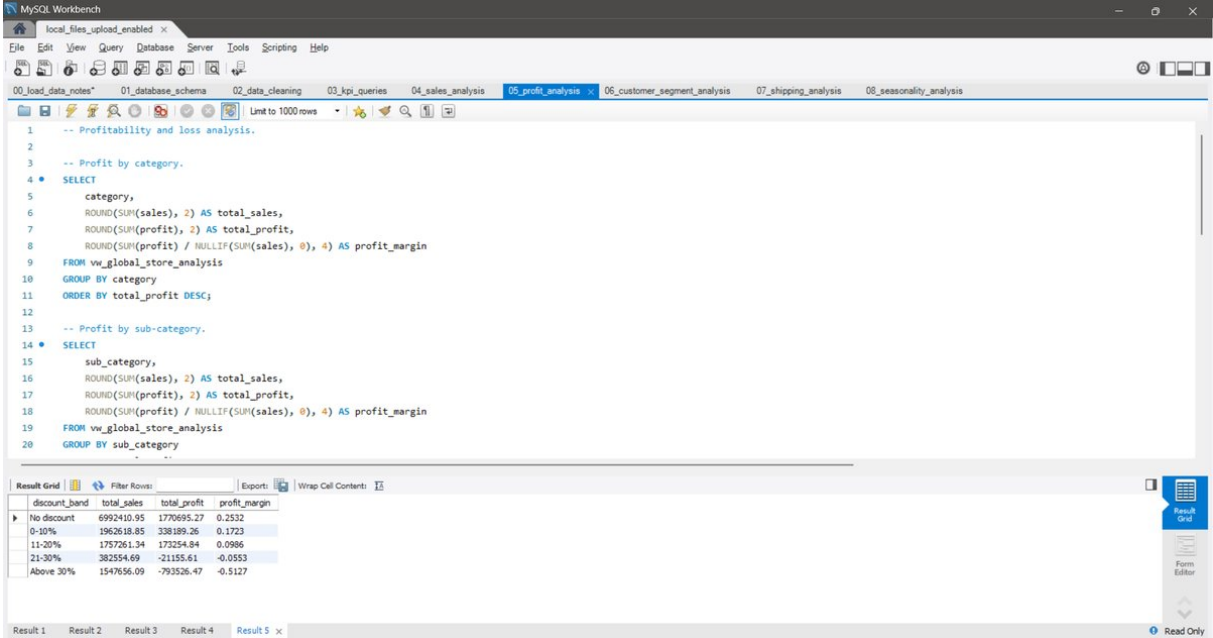
Read Only

MySQL Workbench — monthly sales trend

5.5 Profit & Discount Analysis

This is the most business-critical query set — it isolates exactly where margin is being lost:

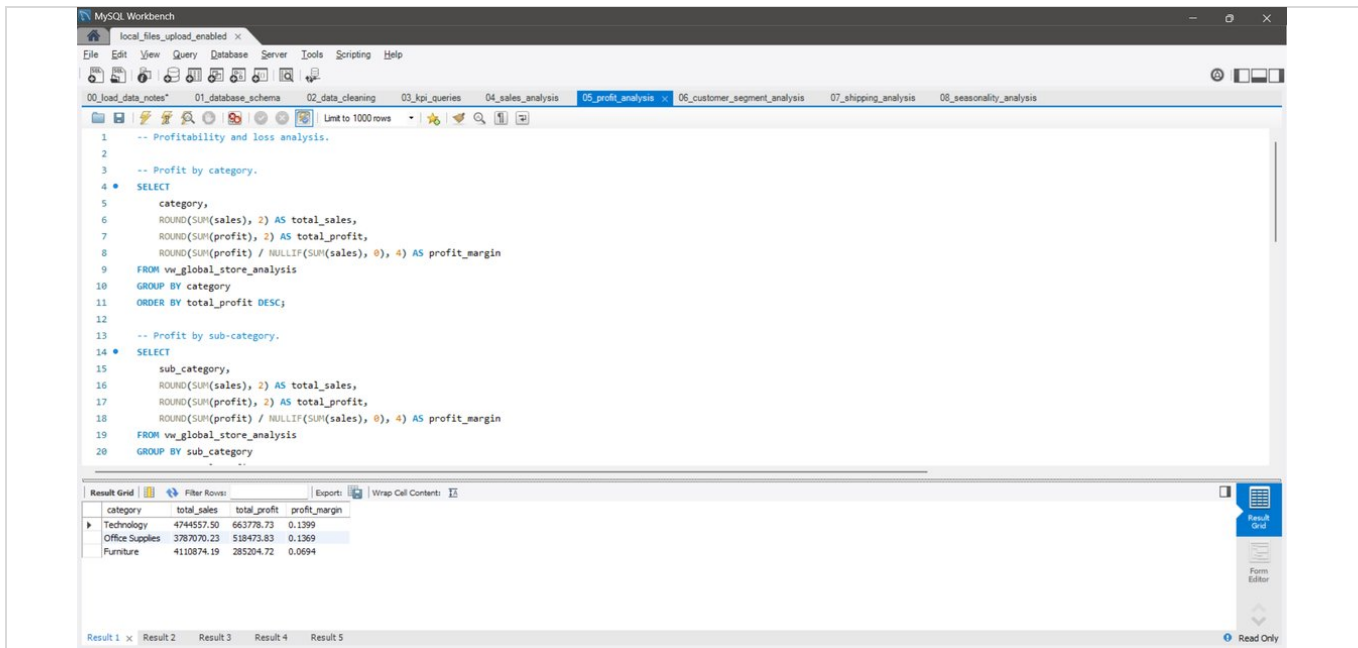
```
-- Discount impact on profit
SELECT
  CASE
    WHEN discount = 0 THEN 'No discount'
    WHEN discount <= 0.10 THEN '0-10%'
    WHEN discount <= 0.20 THEN '11-20%'
    WHEN discount <= 0.30 THEN '21-30%'
    ELSE 'Above 30%'
  END AS discount_band,
  ROUND(SUM(sales), 2) AS total_sales,
  ROUND(SUM(profit), 2) AS total_profit,
  ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin
FROM vw_global_store_analysis
GROUP BY discount_band
ORDER BY MIN(discount);
```



The screenshot shows the MySQL Workbench interface with a query editor and a results grid. The query is a SELECT statement that calculates profit margin by discount band. The results grid shows the following data:

discount_band	total_sales	total_profit	profit_margin
No discount	6992410.95	1770695.27	0.2532
0-10%	1962618.85	238189.26	0.1213
11-20%	1757261.34	172254.04	0.0986
21-30%	382554.69	-21155.61	-0.0553
Above 30%	1547656.09	-793526.47	-0.5127

MySQL Workbench — profit margin collapses as discount band increases



MySQL Workbench

local_files_upload_enabled x

File Edit View Query Database Server Tools Scripting Help

00_load_data_notes 01_database_schema 02_data_cleaning 03_kpi_queries 04_sales_analysis 05_profit_analysis 06_customer_segment_analysis 07_shipping_analysis 08_seasonality_analysis

Limit to 1000 rows

```

1 -- Profitability and loss analysis.
2
3 -- Profit by category.
4 * SELECT
5     category,
6     ROUND(SUM(sales), 2) AS total_sales,
7     ROUND(SUM(profit), 2) AS total_profit,
8     ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin
9 FROM vw_global_store_analysis
10 GROUP BY category
11 ORDER BY total_profit DESC;
12
13 -- Profit by sub-category.
14 * SELECT
15     sub_category,
16     ROUND(SUM(sales), 2) AS total_sales,
17     ROUND(SUM(profit), 2) AS total_profit,
18     ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin
19 FROM vw_global_store_analysis
20 GROUP BY sub_category

```

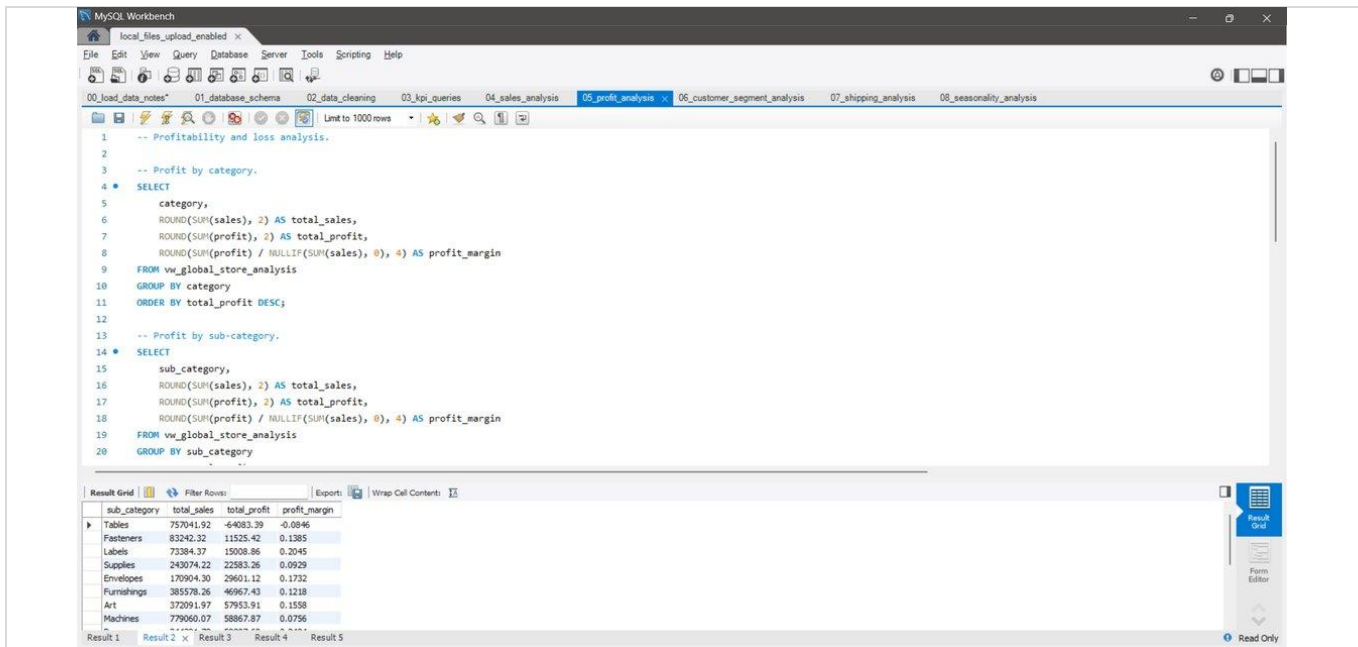
Result Grid

category	total_sales	total_profit	profit_margin
Technology	4744557.50	663778.73	0.1399
Office Supplies	3787070.23	518473.83	0.1369
Furniture	4110874.19	285204.72	0.0694

Result 1 x Result 2 Result 3 Result 4 Result 5

Read Only

MySQL Workbench — profit by category



MySQL Workbench

local_files_upload_enabled x

File Edit View Query Database Server Tools Scripting Help

00_load_data_notes 01_database_schema 02_data_cleaning 03_kpi_queries 04_sales_analysis 05_profit_analysis 06_customer_segment_analysis 07_shipping_analysis 08_seasonality_analysis

Limit to 1000 rows

```

1 -- Profitability and loss analysis.
2
3 -- Profit by category.
4 * SELECT
5     category,
6     ROUND(SUM(sales), 2) AS total_sales,
7     ROUND(SUM(profit), 2) AS total_profit,
8     ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin
9 FROM vw_global_store_analysis
10 GROUP BY category
11 ORDER BY total_profit DESC;
12
13 -- Profit by sub-category.
14 * SELECT
15     sub_category,
16     ROUND(SUM(sales), 2) AS total_sales,
17     ROUND(SUM(profit), 2) AS total_profit,
18     ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin
19 FROM vw_global_store_analysis
20 GROUP BY sub_category

```

Result Grid

sub_category	total_sales	total_profit	profit_margin
Tables	757041.92	-64083.39	-0.0846
Fasteners	83242.32	11525.42	0.1385
Labels	73384.37	15008.86	0.2045
Supplies	243074.22	22383.26	0.0929
Envelopes	170904.30	29601.12	0.1732
Furnishings	385578.26	46967.43	0.1218
Art	372091.97	57953.91	0.1558
Machines	779060.07	58867.87	0.0756

Result 1 Result 2 x Result 3 Result 4 Result 5

Read Only

MySQL Workbench — profit by sub-category (Tables is the clear outlier)

MySQL Workbench — individual products with negative total profit

product_name	category	sub_category	total_sales	total_profit	profit_margin
Cubify Cubex 3D Printer Double Head Print	Technology	Machines	11099.96	-879.97	-0.8000
Lexmark MX611dhe Monochrome Laser Printer	Technology	Machines	16829.90	-4589.97	-0.2727
Motorola Smart Phone, Cordless	Technology	Phones	38931.04	-4447.04	-0.1142
Bevis Conference Table, Fully Assembled	Furniture	Tables	24931.68	-4377.49	-0.1752
Cubify Cubex 3D Printer Triple Head Print	Technology	Machines	7999.98	-3839.99	-0.4800
Bevis Computer Table, Fully Assembled	Furniture	Tables	11177.90	-3509.56	-0.3140
Rogers Lockers, Blue	Office Supplies	Storage	28214.59	-2893.49	-0.1026
Chromcraft Bull-Nose Wood Oval Conference T...	Furniture	Tables	9917.64	-2876.12	-0.2900

MySQL Workbench — individual products with negative total profit

5.6 Customer & Segment Analysis

```

SELECT
    segment,
    ROUND(SUM(sales), 2) AS total_sales,
    ROUND(SUM(profit), 2) AS total_profit,
    ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin,
    COUNT(DISTINCT customer_id) AS customers
FROM vw_global_store_analysis
GROUP BY segment
ORDER BY total_sales DESC;

```

MySQL Workbench — sales, profit and margin by customer segment

segment	total_sales	total_profit	profit_margin	customers
Consumer	6492506.86	746261.47	0.1149	818
Corporate	3845641.02	444951.83	0.1157	475
Home Office	2304354.03	276244.00	0.1199	296

MySQL Workbench — sales, profit and margin by customer segment

MySQL Workbench interface showing a query for top 10 customers by sales. The query is executed, and the result grid displays the following data:

customer_id	customer_name	segment	total_sales	total_profit
TA-21385	Tom Ashbrook	Home Office	36663.12	6328.05
TC-20980	Tamara Chand	Corporate	36175.36	9392.65
GT-14710	Greg Tran	Consumer	36099.41	5408.73
SM-20320	Sean Miller	Home Office	31148.85	-1079.20
PH-14365	Fred Hopkins	Corporate	30429.22	4478.56
PS-18045	Penelope Sewall	Home Office	30070.00	4317.89
BW-11110	Bert Walters	Corporate	29860.86	3302.07
HL-15040	Hunter Lopez	Consumer	29688.23	7658.46
SE-20110	Sanjit Engle	Consumer	29584.30	5886.59
RB-19360	Raymond Buch	Consumer	29197.63	8523.95

MySQL Workbench — top 10 customers by sales

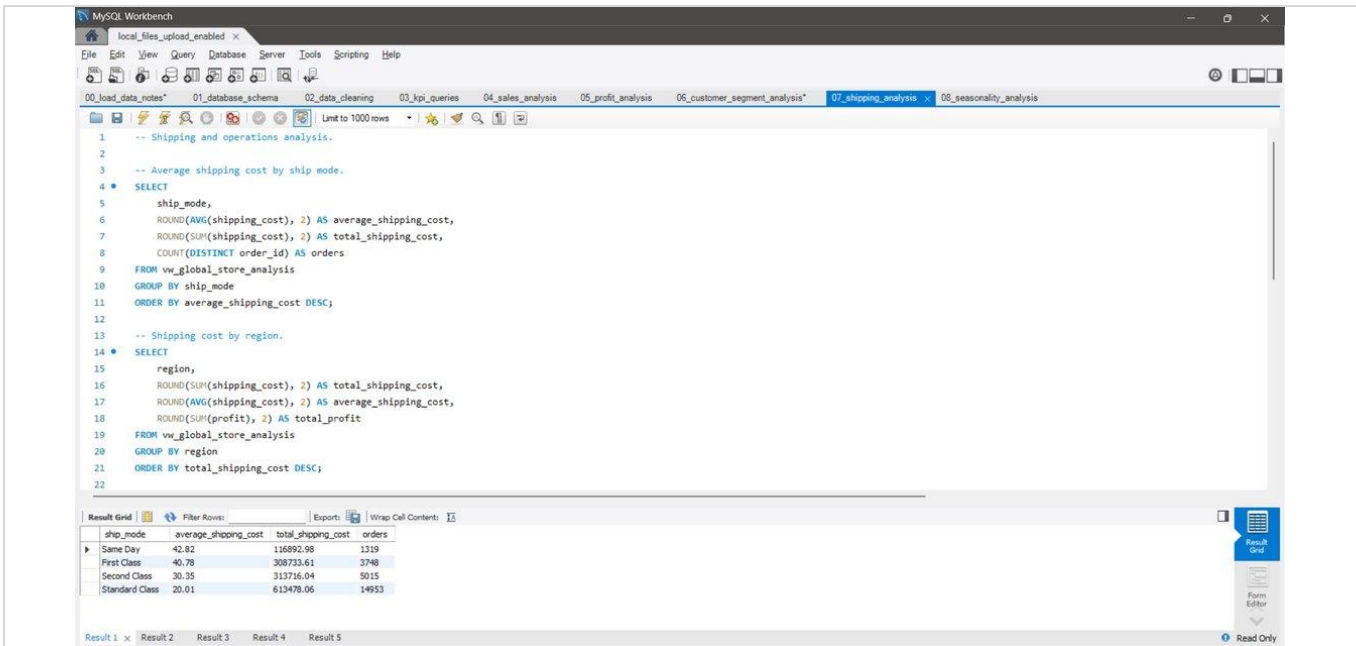
MySQL Workbench interface showing a query for region and segment performance. The query is executed, and the result grid displays the following data:

region	segment	total_sales	total_profit	profit_margin
Africa	Consumer	424198.19	47716.55	0.1125
Africa	Corporate	204661.36	20364.13	0.0995
Africa	Home Office	154913.66	20790.95	0.1342
Canada	Consumer	35117.94	9557.16	0.2721
Canada	Corporate	19589.79	5056.44	0.2581
Canada	Home Office	12220.44	3203.79	0.2622
Caribbean	Consumer	162891.22	20083.58	0.1234
Caribbean	Corporate	104615.86	9154.70	0.0875
Caribbean	Home Office	57144.61	5499.67	0.0962
Central	Consumer	1471825.57	149012.43	0.1012
Central	Corporate	857976.43	104810.83	0.1222

MySQL Workbench — region x segment performance

5.7 Shipping & Operations Analysis

```
SELECT
    ship_mode,
    ROUND(SUM(sales), 2) AS total_sales,
    ROUND(SUM(profit), 2) AS total_profit,
    ROUND(SUM(shipping_cost), 2) AS total_shipping_cost,
    ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin
FROM vw_global_store_analysis
GROUP BY ship_mode
ORDER BY profit_margin ASC;
```



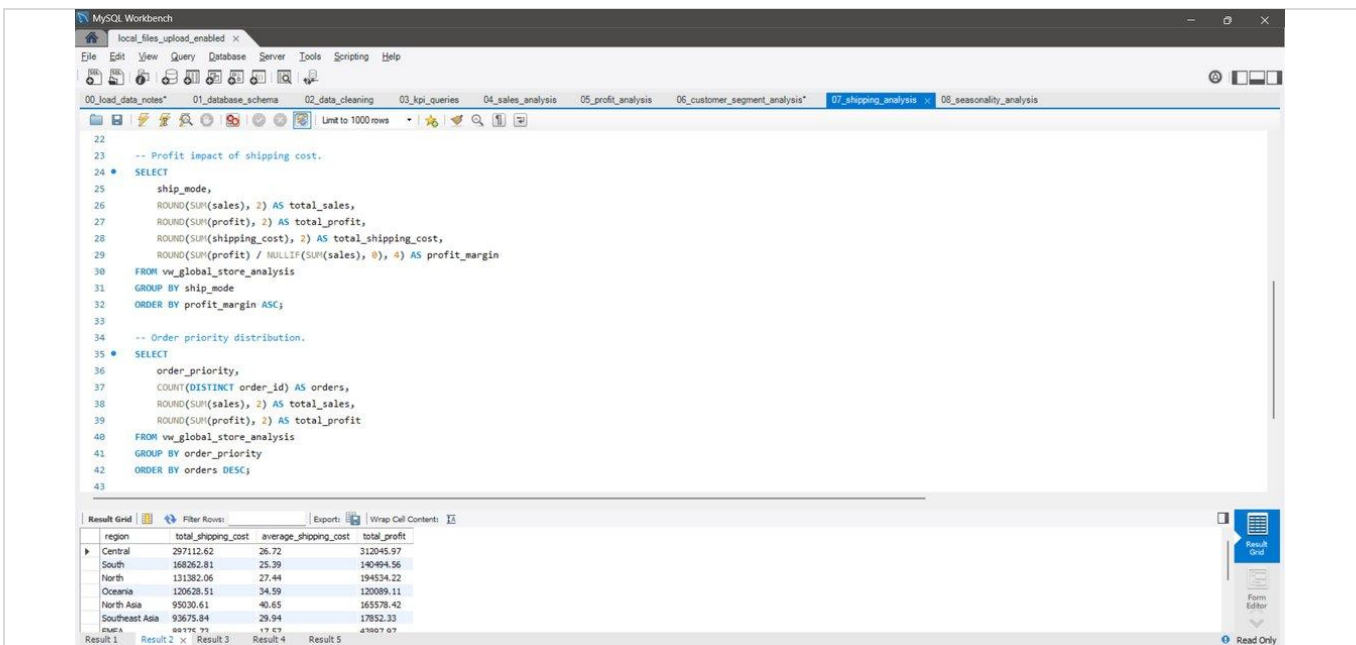
The screenshot shows the MySQL Workbench interface with a query window titled '07_shipping_analysis'. The query is as follows:

```
-- Shipping and operations analysis.
-- Average shipping cost by ship mode.
SELECT
    ship_mode,
    ROUND(AVG(shipping_cost), 2) AS average_shipping_cost,
    ROUND(SUM(shipping_cost), 2) AS total_shipping_cost,
    COUNT(DISTINCT order_id) AS orders
FROM vw_global_store_analysis
GROUP BY ship_mode
ORDER BY average_shipping_cost DESC;
```

The results are displayed in a table with the following data:

ship_mode	average_shipping_cost	total_shipping_cost	orders
Same Day	42.82	116992.98	1319
First Class	40.78	308733.61	3748
Second Class	30.35	313735.04	3015
Standard Class	20.01	613478.06	14953

MySQL Workbench — average shipping cost by ship mode



The screenshot shows the MySQL Workbench interface with a query window titled '07_shipping_analysis'. The query is as follows:

```
-- Profit impact of shipping cost.
SELECT
    ship_mode,
    ROUND(SUM(sales), 2) AS total_sales,
    ROUND(SUM(profit), 2) AS total_profit,
    ROUND(SUM(shipping_cost), 2) AS total_shipping_cost,
    ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin
FROM vw_global_store_analysis
GROUP BY ship_mode
ORDER BY profit_margin ASC;
```

The results are displayed in a table with the following data:

ship_mode	total_sales	total_profit	total_shipping_cost	profit_margin
Same Day	116992.98	1319	42.82	0.1126
First Class	308733.61	3748	40.78	0.1217
Second Class	313735.04	3015	30.35	0.0961
Standard Class	613478.06	14953	20.01	0.0326

MySQL Workbench — profit margin by ship mode

MySQL Workbench interface showing two SQL queries and their results.

Query 1: Profit impact of shipping cost.

```

22
23 -- Profit impact of shipping cost.
24 * SELECT
25     ship_mode,
26     ROUND(SUM(sales), 2) AS total_sales,
27     ROUND(SUM(profit), 2) AS total_profit,
28     ROUND(SUM(shipping_cost), 2) AS total_shipping_cost,
29     ROUND(SUM(profit) / NULLIF(SUM(sales), 0), 4) AS profit_margin
30 FROM vw_global_store_analysis
31 GROUP BY ship_mode
32 ORDER BY profit_margin ASC;

```

ship_mode	total_sales	total_profit	total_shipping_cost	profit_margin
Medium	14281	7239521.30	859055.65	
High	7638	3830787.54	422353.37	
Critical	1941	999682.48	126039.35	
Low	1175	572540.59	60008.92	

Query 2: Order priority distribution.

```

34 -- Order priority distribution.
35 * SELECT
36     order_priority,
37     COUNT(DISTINCT order_id) AS orders,
38     ROUND(SUM(sales), 2) AS total_sales,
39     ROUND(SUM(profit), 2) AS total_profit
40 FROM vw_global_store_analysis
41 GROUP BY order_priority
42 ORDER BY orders DESC;

```

order_priority	orders	total_sales	total_profit
Medium	14281	7239521.30	859055.65
High	7638	3830787.54	422353.37
Critical	1941	999682.48	126039.35
Low	1175	572540.59	60008.92

MySQL Workbench — average delivery days by ship mode

5.8 Seasonality Analysis

```

SELECT order_year, order_quarter,
       ROUND(SUM(sales), 2) AS quarterly_sales,
       ROUND(SUM(profit), 2) AS quarterly_profit
FROM vw_global_store_analysis
GROUP BY order_year, order_quarter
ORDER BY quarterly_sales DESC;

```

MySQL Workbench interface showing a SQL query and its results.

Query: Seasonality and trend analysis.

```

1 -- Seasonality and trend analysis.
2
3 -- Monthly sales trend.
4 * SELECT
5     order_year,
6     order_month,
7     ROUND(SUM(sales), 2) AS monthly_sales
8 FROM vw_global_store_analysis
9 GROUP BY order_year, order_month
10 ORDER BY order_year, order_month;

```

order_year	order_month	monthly_sales
2011	1	99905.61
2011	2	90353.89
2011	3	145115.69
2011	4	118317.65
2011	5	144176.13
2011	6	215879.58
2011	7	113706.72
2011	8	207013.39
2011	9	290324.43
2011	10	200075.31
2011	11	300822.80
2011	12	333799.69
2012	1	134276.46
2012	2	101200.57
2012	3	165581.60
2012	4	161026.88

MySQL Workbench — monthly sales trend

MySQL Workbench interface showing a query in the editor and its result grid. The query is a SELECT statement grouped by order_year and order_month, ordered by quarterly_sales in descending order. The result grid displays 12 rows of data for the years 2011 and 2012.

```

19 GROUP BY order_year, order_month
20 ORDER BY order_year, order_month;
21
22 -- Best quarter by sales.
23 * SELECT
24     order_year,
25     order_quarter,
26     ROUND(SUM(sales), 2) AS quarterly_sales,
27     ROUND(SUM(profit), 2) AS quarterly_profit
28 FROM vw_global_store_analysis
29 GROUP BY order_year, order_quarter
30 ORDER BY quarterly_sales DESC;
31
32 -- Best month by category.

```

order_year	order_month	monthly_sales
2011	1	99905.61
2011	2	90353.89
2011	3	145115.69
2011	4	118317.65
2011	5	144176.13
2011	6	215879.58
2011	7	113706.72
2011	8	207013.39
2011	9	290324.43
2011	10	200075.31
2011	11	300822.80
2011	12	333759.69
2012	1	134276.46
2012	2	101200.57
2012	3	165581.60
2012	4	161026.88

MySQL Workbench — best quarters by sales

MySQL Workbench interface showing a query in the editor and its result grid. The query is a SELECT statement grouped by category and order_month, ordered by total_sales in descending order. The result grid displays 12 rows of data for various categories including Furniture, Office S..., and Office S....

```

28 FROM vw_global_store_analysis
29 GROUP BY order_year, order_quarter
30 ORDER BY quarterly_sales DESC;
31
32 -- Best month by category.
33 * SELECT
34     category,
35     order_month,
36     ROUND(SUM(sales), 2) AS total_sales
37 FROM vw_global_store_analysis
38 GROUP BY category, order_month
39 ORDER BY category, total_sales DESC;
40

```

category	order_month	total_sales
Furniture	12	538107.79
Furniture	11	504114.72
Furniture	9	491048.63
Furniture	6	412928.84
Furniture	8	392042.25
Furniture	10	380793.17
Furniture	5	278112.84
Furniture	3	255502.98
Furniture	1	231490.19
Furniture	7	227882.80
Furniture	4	211357.74
Furniture	2	187462.23
Office S...	11	469912.01
Office S...	12	467518.84
Office S...	9	427420.63
Office S...	8	399854.62

MySQL Workbench — seasonality summary output

6. Power BI Dashboard

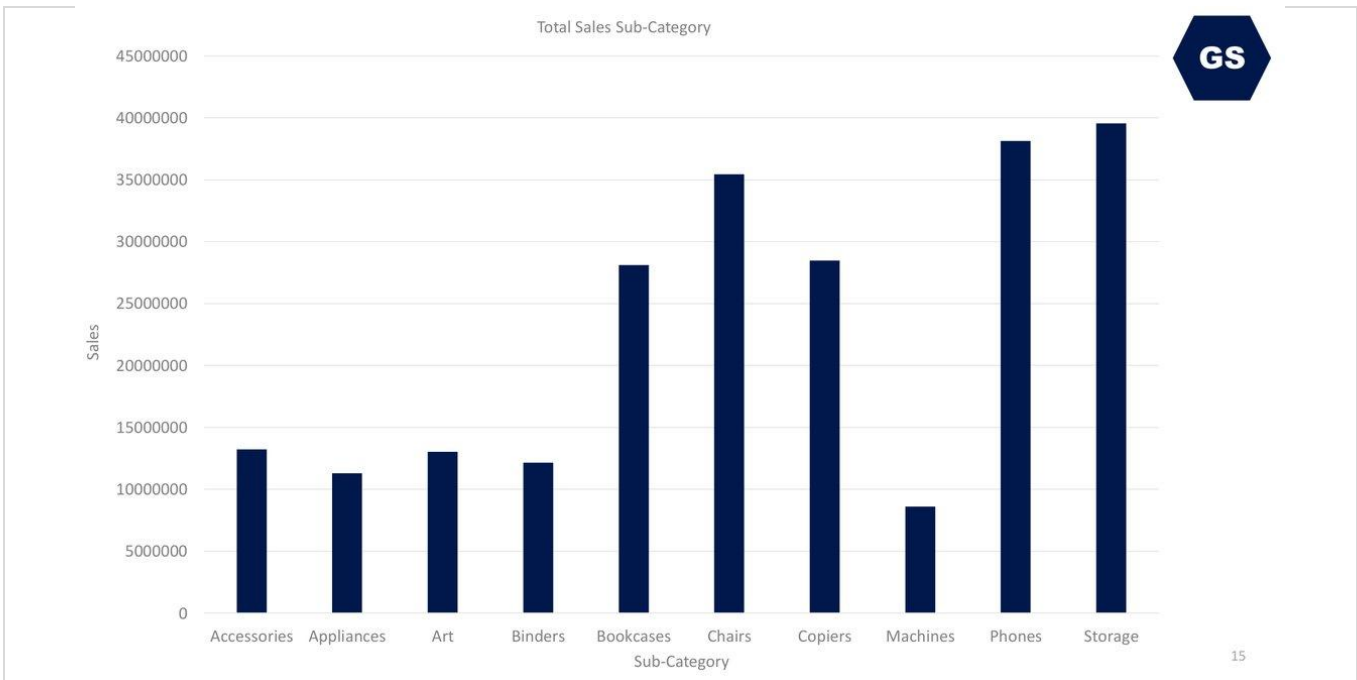
The SQL and Python outputs feed a five-page Power BI dashboard built for management review: an executive overview plus dedicated pages for sales, profitability, customers, and shipping.

6.1 Executive Overview



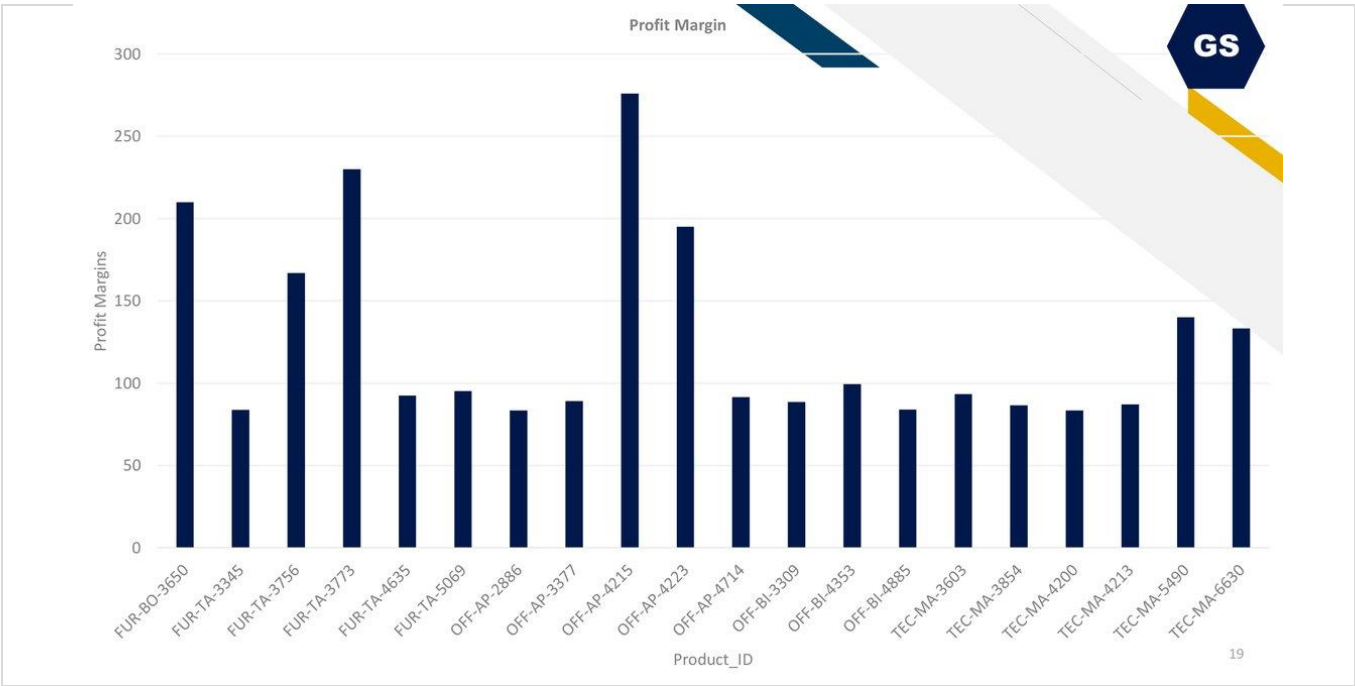
Power BI — executive overview: sales, profit, margin, orders, and trend at a glance

6.2 Sales Performance



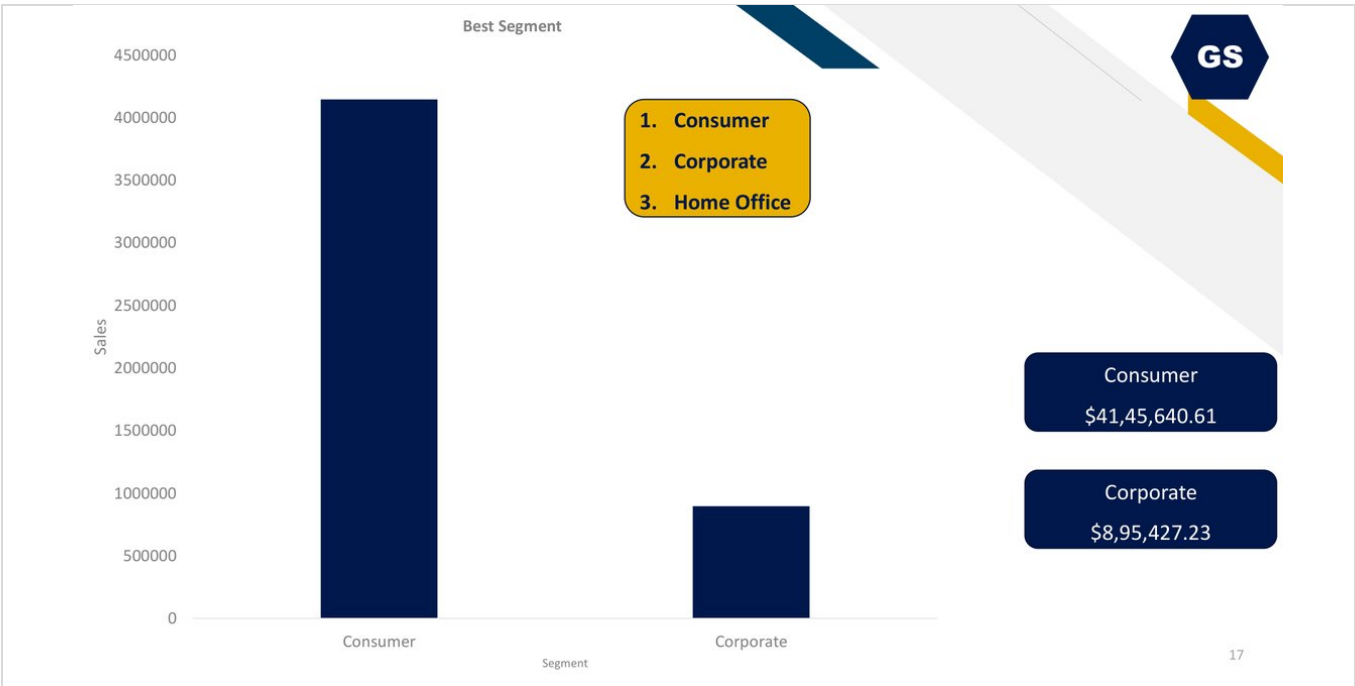
Power BI — sales performance by product, category, region, and market

6.3 Profitability Analysis



Power BI — profit margin, loss-making products, and discount impact

6.4 Customer & Segment Analysis



Power BI — segment contribution, top customers, region-segment performance

6.5 Shipping & Operations



Power BI — ship mode cost, delivery days, and order priority mix

7. Key Findings

7.1 Sales scale is strong, but margin needs control

Global Store produced \$12.64M in sales and \$1.47M in profit, an overall margin of 11.61%. Profit varies sharply by product group, discount level, and region.

7.2 Technology is the strongest category

Technology generated ~\$4.74M in sales and ~\$663.78K in profit — the strongest category on both revenue and profit contribution.

7.3 Furniture needs margin review

Furniture generated ~\$4.11M in sales but only ~\$285.20K in profit, a much weaker margin profile than Technology and Office Supplies.

7.4 Tables is the main loss-making sub-category

Tables generated ~\$757K in sales but -\$64K in profit — the clearest product-level issue in the dataset, needing review across pricing, discounting, supplier cost, and shipping cost.

7.5 Deep discounts destroy profit

No-discount orders produce ~25.32% margin; orders discounted above 30% produce ~-51.27% margin. This is the strongest evidence in the dataset that discount policy needs tighter control.

7.6 Southeast Asia: high sales, weak profitability

Southeast Asia generated ~\$884K in sales but only ~\$17.9K in profit, margin near 2.02%. Sales growth in this region needs a margin-improvement plan before further investment.

7.7 Customer segments play different roles

Consumer customers generate the most revenue; Home Office has the strongest margin — Consumer for scale, Home Office for margin-focused growth.

8. Business Recommendations

Priority	Action	Expected Impact
1	Review Tables pricing, discounting, supplier cost, and shipping cost	Stop recurring loss from a high-sales sub-category
2	Add control rules for discounts above 30%	Reduce margin erosion
3	Investigate Southeast Asia product mix, discounting, and shipping cost	Improve regional profit quality
4	Push profitable Technology products through targeted campaigns	Grow revenue without weakening margin
5	Build segment campaigns around Consumer scale and Home Office margin	Improve customer value and retention

Priority	Action	Expected Impact
6	Keep Power BI KPI monitoring active for sales, profit, and margin	Make performance issues visible earlier

9. Conclusion

This project demonstrates a complete analytics workflow — Python for data preparation, MySQL for relational modelling and business querying, and Power BI for executive reporting — applied to a real commercial question set. The evidence is consistent across all three layers: Global Store's growth is not limited by sales volume, but by margin discipline in the Tables sub-category, discount policy above 30%, and profit quality in Southeast Asia. Acting on the six recommendations above would materially improve profitable growth without sacrificing revenue scale.

Caveats

- The dataset covers 2011–2014 transactions and does not reflect current pricing or cost structures.
- The dataset source and usage terms belong to the Kaggle dataset owner.

— End of Report —